

Aula 10 — Aprendizado rápido II

A cota de arrependimento do UCB • PAC e bandits bayesianos • O problema do alinhamento de valores

Prof. Marcelo Keese Albertini • Universidade Federal de Uberlândia

Material adaptado e expandido de CS234: *Reinforcement Learning*, Emma Brunskill (Stanford), *Lecture 10*, com palestra convidada sobre alinhamento por Wanheng Hu. Referência técnica principal: Lattimore & Szepesvári, *Bandit Algorithms* (2020), seção 7.1.

Como usar este resumo

Este texto acompanha os slides da Aula 10 e é mais completo que eles em três pontos: a prova da cota do UCB aparece com *todas* as passagens algébricas, há uma seção de exercícios com respostas comentadas, e a discussão sobre alinhamento vem organizada como uma taxonomia que você pode usar para classificar casos novos.

A aula tem duas metades de natureza muito diferente. A primeira é a mais técnica do curso até aqui; a segunda, a menos técnica. O elo entre elas está no último parágrafo da Seção 6: uma cota de arrependimento garante que otimizamos *bem* o objetivo que escrevemos, e é completamente silenciosa sobre esse objetivo ser o correto.

1 O problema e o critério

Definição — bandit multiarmado

Um par $(\mathcal{A}, \mathcal{D})$, com $\mathcal{A}$ um conjunto *conhecido* de K ações (“braços”) e $\mathcal{D}_a(r) = \mathbb{P}[r \mid a]$ distribuições *desconhecidas* de recompensa. A cada passo t o agente escolhe $a_t \in \mathcal{A}$ e o ambiente devolve $r_t \sim \mathcal{D}_{a_t}$, independentemente do passado dado a_t .

Notação usada em todo o texto:

- ▶ $Q(a) = \mathbb{E}[r \mid a]$ — média verdadeira (desconhecida) do braço a ;
- ▶ $V^* = \max_a Q(a)$, com braço ótimo a_1 (sem perda de generalidade);
- ▶ $\Delta_i = V^* - Q(a_i) \geq 0$ — a **lacuna** do braço i ;
- ▶ $N_t(a)$ — número de puxadas de a até o instante t ;
- ▶ $\hat{Q}_t(a)$ — média empírica das recompensas observadas em a ;
- ▶ $\hat{\mu}_{i,s}$ — média das s primeiras recompensas do braço i .

Definição — arrependimento

$\ell_t = \mathbb{E}[V^* - Q(a_t)]$ (um passo) e $L_n = \mathbb{E}[\sum_{t=1}^n (V^* - Q(a_t))]$ (acumulado em n passos).

Como nV^* não depende do algoritmo, *maximizar recompensa acumulada é exatamente o mesmo problema que minimizar arrependimento*. A pergunta interessante não é se L_n é pequeno — ele cresce com n em qualquer algoritmo — e sim *como* cresce. Arrependimento sublinear ($L_n/n \rightarrow 0$) significa que o agente acaba jogando quase sempre o braço ótimo; arrependimento linear significa que erra uma fração constante do tempo, para sempre.

Decomposição do arrependimento por braço

Para qualquer algoritmo e qualquer horizonte n ,

$$L_n = \sum_{i=1}^K \Delta_i \mathbb{E}[N_n(a_i)].$$

Demonstração

Cada passo t contribui com $V^* - Q(a_t)$, e há exatamente $N_n(a_i)$ passos em que $a_t = a_i$. Como $\sum_i N_n(a_i) = n$,

$$\sum_{t=1}^n (V^* - Q(a_t)) = \sum_{i=1}^K N_n(a_i) (V^* - Q(a_i)) = \sum_{i=1}^K \Delta_i N_n(a_i).$$

Tomando esperança dos dois lados obtém-se o resultado.

Intuição

Esta identidade é o organizador de toda a análise: ela converte um problema sobre *recompensas* num problema sobre *contagens*. Provar uma cota de arrependimento passa a ser uma única tarefa — limitar $\mathbb{E}[N_n(a_i)]$ para cada braço subótimo. Braços péssimos (Δ_i grande) são fáceis de descartar; o trabalho está nos braços *quase* tão bons quanto o ótimo.

2 A ferramenta: concentração**Definição — variável σ -subgaussiana**

X com $\mathbb{E}[X] = 0$ é σ -subgaussiana se $\mathbb{E}[e^{\lambda X}] \leq e^{\lambda^2 \sigma^2 / 2}$ para todo $\lambda \in \mathbb{R}$. Recompensas limitadas em $[0, 1]$ são $\frac{1}{2}$ -subgaussianas (Hoeffding); ruído $\mathcal{N}(0, \sigma^2)$ é σ -subgaussiano. Assumimos $\sigma = 1$ para não carregar constantes.

Cota de concentração (a única que será usada)

Se $\hat{\mu}_s$ é a média de s amostras i.i.d. de média μ com ruído 1-subgaussiano, então para todo $\varepsilon > 0$

$$\mathbb{P}(\hat{\mu}_s \geq \mu + \varepsilon) \leq e^{-s\varepsilon^2/2}, \quad \mathbb{P}(\hat{\mu}_s \leq \mu - \varepsilon) \leq e^{-s\varepsilon^2/2}.$$

Fazendo $\delta = e^{-s\varepsilon^2/2}$, isto é $\varepsilon = \sqrt{2 \log(1/\delta)/s}$: com probabilidade ao menos $1 - \delta$, $|\hat{\mu}_s - \mu| < \sqrt{2 \log(1/\delta)/s}$.

A segunda forma é uma **barra de erro**: após s amostras, a média verdadeira dista menos de $\sqrt{2 \log(1/\delta)/s}$ da estimativa, salvo com probabilidade δ . É exatamente esse comprimento que o UCB usa como bônus — e nada além disso será usado na prova.

Algoritmo — UCB(δ)

1. Puxe cada braço uma vez.
2. Para $t = K + 1, \dots, n$, escolha

$$a_t = \arg \max_{a \in \mathcal{A}} \text{UCB}_a(t-1, \delta), \quad \text{UCB}_a(t, \delta) = \hat{Q}_t(a) + \sqrt{\frac{2 \log(1/\delta)}{N_t(a)}}.$$

3. Observe r_t ; atualize $\hat{Q}_t(a_t)$ e $N_t(a_t)$.

Duas leituras do índice

O índice UCB_a é (i) o extremo superior de um intervalo de confiança de nível $1 - \delta$ para $Q(a)$; e (ii) o valor que o braço teria *no melhor dos mundos ainda plausíveis* — daí o nome “otimismo diante da incerteza”.

Por que otimismo funciona: se o braço escolhido é de fato bom, jogá-lo é barato; se é ruim, jogá-lo revela isso depressa e o índice cai. Não existe um terceiro desfecho em que o algoritmo se prenda indefinidamente a um braço ruim — exatamente o desfecho possível com um bônus *constante*, que não distingue o que é pouco conhecido do que é bem conhecido.

3 A cota de arrependimento do UCB

Teorema (Lattimore & Szepesvári, Thm. 7.1)

Considere o $UCB(\delta)$ num bandit estocástico de K braços com ruído 1-subgaussiano. Para qualquer horizonte n , com $\delta = 1/n^2$,

$$L_n \leq 3 \sum_{i=1}^K \Delta_i + \sum_{i: \Delta_i > 0} \frac{16 \log n}{\Delta_i}.$$

A prova tem cinco passos. Fixe um braço subótimo a_i e um inteiro u_i (a escolher no Passo 4).

3.1 Passo 1 — o evento bom

Definição — evento bom G_i

$$G_i = \underbrace{\left\{ Q(a_1) < \min_{t \in [n]} UCB_{a_1}(t, \delta) \right\}}_{(*)} \cap \underbrace{\left\{ \hat{\mu}_{i, u_i} + \sqrt{\frac{2 \log(1/\delta)}{u_i}} < Q(a_1) \right\}}_{(**)}$$

Em palavras: (*) diz que *o braço ótimo nunca é subestimado* — seu índice permanece acima da média verdadeira em todos os n passos; (**) diz que *o braço i já foi desmascarado* — depois de u_i puxadas, mesmo somando o bônus, seu índice cai abaixo de $Q(a_1)$.

3.2 Passo 2 — no evento bom, o braço i é puxado no máximo u_i vezes

Afirmção

Se G_i ocorre, então $N_n(a_i) \leq u_i$.

Demonstração

Por contradição. Suponha $N_n(a_i) > u_i$. Então existe um instante t em que o braço i foi escolhido tendo sido puxado exatamente u_i vezes antes, isto é, $N_{t-1}(a_i) = u_i$ e $a_t = a_i$. Nesse instante,

$$UCB_{a_i}(t-1, \delta) = \hat{\mu}_{i, u_i} + \sqrt{\frac{2 \log(1/\delta)}{u_i}} \stackrel{(**)}{<} Q(a_1) \stackrel{(*)}{<} UCB_{a_1}(t-1, \delta).$$

Como o UCB escolhe o *maior* índice, ele teria selecionado a_1 , não a_i — contradição.

Intuição

Este passo é *puramente determinístico*: toda a aleatoriedade foi empurrada para a definição de G_i , e aqui só se usa que o algoritmo é um $\arg \max$. É o padrão de toda análise de bandits — argumentar deterministicamente dentro do evento bom e usar a cota trivial fora dele.

Dividindo a esperança conforme G_i ocorra ou não, e usando $N_n(a_i) \leq n$ no segundo caso:

$$\mathbb{E}[N_n(a_i)] = \mathbb{E}[\mathbf{1}\{G_i\}N_n(a_i)] + \mathbb{E}[\mathbf{1}\{G_i^c\}N_n(a_i)] \leq u_i + n \mathbb{P}(G_i^c). \quad (1)$$

O primeiro termo é o **custo do aprendizado** (quantas amostras bastam para distinguir a_i de a_1); o segundo é o **custo do azar** (se as barras de erro falharem, o algoritmo pode gastar todo o horizonte no braço errado).

3.3 Passo 3 — limitando $\mathbb{P}(G_i^c)$ **(a) A probabilidade de (*) falhar**

Se (*) falha, existe algum t com $Q(a_1) \geq \text{UCB}_{a_1}(t, \delta)$. O número de puxadas do braço 1 nesse instante é algum $s \in \{1, \dots, n\}$, logo

$$\left\{ Q(a_1) \geq \min_{t \in [n]} \text{UCB}_{a_1}(t, \delta) \right\} \subseteq \bigcup_{s=1}^n \left\{ Q(a_1) \geq \hat{\mu}_{1,s} + \sqrt{\frac{2 \log(1/\delta)}{s}} \right\}.$$

Pela cota da união e pela concentração com $\varepsilon = \sqrt{2 \log(1/\delta)/s}$:

$$\mathbb{P}((*) \text{ falha}) \leq \sum_{s=1}^n \mathbb{P}\left(\hat{\mu}_{1,s} \leq Q(a_1) - \sqrt{\frac{2 \log(1/\delta)}{s}}\right) \leq \sum_{s=1}^n e^{-s \cdot \frac{2 \log(1/\delta)}{2s}} = n\delta. \quad (2)$$

Note o cancelamento: o expoente $s\varepsilon^2/2$ elimina exatamente o s do denominador, de modo que *cada* barra de erro falha com probabilidade δ , e pagamos o fator n por exigir que todas valham simultaneamente. É essa exigência uniforme em t que obrigará δ a ser da ordem de n^{-2} .

(b) A probabilidade de () falhar**

Suponha que u_i seja grande o bastante para que, para algum $c \in (0, 1)$,

$$\Delta_i - \sqrt{\frac{2 \log(1/\delta)}{u_i}} \geq c \Delta_i. \quad (3)$$

Usando $Q(a_1) = Q(a_i) + \Delta_i$:

$$\begin{aligned} \mathbb{P}((**) \text{ falha}) &= \mathbb{P}\left(\hat{\mu}_{i,u_i} + \sqrt{\frac{2 \log(1/\delta)}{u_i}} \geq Q(a_i) + \Delta_i\right) \\ &= \mathbb{P}\left(\hat{\mu}_{i,u_i} - Q(a_i) \geq \Delta_i - \sqrt{\frac{2 \log(1/\delta)}{u_i}}\right) \\ &\leq \mathbb{P}\left(\hat{\mu}_{i,u_i} - Q(a_i) \geq c \Delta_i\right) && \text{por (3)} \\ &\leq \exp\left(-\frac{u_i c^2 \Delta_i^2}{2}\right) && \text{concentração.} \end{aligned}$$

Somando com (2) e substituindo em (1):

$$\mathbb{E}[N_n(a_i)] \leq u_i + n \left(n\delta + \exp\left(-\frac{u_i c^2 \Delta_i^2}{2}\right) \right). \quad (4)$$

3.4 Passo 4 — escolhendo u_i

A condição (3) é uma desigualdade em u_i . Resolvendo-a:

$$(1-c)\Delta_i \geq \sqrt{\frac{2 \log(1/\delta)}{u_i}} \iff (1-c)^2 \Delta_i^2 \geq \frac{2 \log(1/\delta)}{u_i} \iff u_i \geq \frac{2 \log(1/\delta)}{(1-c)^2 \Delta_i^2}.$$

Tomamos o menor inteiro admissível,

$$u_i = \left\lceil \frac{2 \log(1/\delta)}{(1-c)^2 \Delta_i^2} \right\rceil \leq 1 + \frac{2 \log(1/\delta)}{(1-c)^2 \Delta_i^2},$$

o que permite estimar o termo exponencial de (4):

$$\exp\left(-\frac{u_i c^2 \Delta_i^2}{2}\right) \leq \exp\left(-\frac{c^2}{(1-c)^2} \log \frac{1}{\delta}\right) = \delta^{c^2/(1-c)^2}.$$

Portanto

$$\mathbb{E}[N_n(a_i)] \leq 1 + \frac{2 \log(1/\delta)}{(1-c)^2 \Delta_i^2} + n^2 \delta + n \delta^{c^2/(1-c)^2}. \quad (5)$$

3.5 Passo 5 — fechando as contas

Escolha $c = \frac{1}{2}$ e $\delta = 1/n^2$. Então $c^2/(1-c)^2 = 1$, $(1-c)^2 = \frac{1}{4}$ e $\log(1/\delta) = 2 \log n$, de modo que os termos de (5) viram

$$\frac{2 \cdot 2 \log n}{\frac{1}{4} \Delta_i^2} = \frac{16 \log n}{\Delta_i^2}, \quad n^2 \delta + n \delta = 1 + \frac{1}{n} \leq 2.$$

Logo, para todo braço subótimo,

$$\mathbb{E}[N_n(a_i)] \leq 3 + \frac{16 \log n}{\Delta_i^2}$$

e, pela decomposição por braço,

$$L_n = \sum_i \Delta_i \mathbb{E}[N_n(a_i)] \leq 3 \sum_{i=1}^K \Delta_i + \sum_{i: \Delta_i > 0} \frac{16 \log n}{\Delta_i}. \quad \blacksquare$$

A constante 3 é 1 (do teto em u_i) somado a 2 (do custo do azar); ela não depende de n .

Para discutir em aula

Onde exatamente a prova usa que o problema é *estacionário*? E se as recompensas tivessem caudas pesadas (variância infinita), qual dos cinco passos quebraria primeiro?

4 Como ler a cota

$$L_n \leq \underbrace{3 \sum_{i=1}^K \Delta_i}_{\text{constante em } n} + \underbrace{\sum_{i: \Delta_i > 0} \frac{16 \log n}{\Delta_i}}_{\text{cresce como } \log n}$$

► **Em n :** crescimento $O(\log n)$ — extraordinariamente lento. Com $n = 10^6$, $\log n \approx 14$. Em particular $L_n/n \rightarrow 0$.

- ▶ **Em Δ_i :** cada braço subótimo custa $\approx 16 \log n / \Delta_i$. Braços *péssimos* custam pouco, porque são identificados em poucas amostras.
- ▶ **Em K :** linear, escondida no somatório.
- ▶ **O que a cota não diz:** nada sobre a variância do arrependimento nem sobre qualquer execução individual. É uma afirmação sobre a média.

Atenção

A expressão *explode* quando $\Delta_i \rightarrow 0$, o que é um artefato da forma como a escrevemos e não do algoritmo: se Δ_i é minúsculo, puxar o braço i quase não custa nada. É preciso uma segunda leitura.

A tensão: braços quase ótimos

Um braço com lacuna Δ contribui ao arrependimento no máximo $\min\{n\Delta, 16 \log n / \Delta\}$: o primeiro termo porque ele não pode ser puxado mais de n vezes, o segundo pelo teorema. Braços com Δ grande são descartados depressa; braços com Δ minúsculo podem ser puxados quase sempre, mas cada puxada custa quase nada. O pior caso está no meio — Δ grande o bastante para doer, pequeno o bastante para confundir. Formalizando essa observação:

Corolário (cota independente da distribuição)

Para o mesmo $\text{UCB}(1/n^2)$ e *qualquer* instância de K braços,

$$L_n \leq 3 \sum_{i=1}^K \Delta_i + 8\sqrt{Kn \log n}.$$

Demonstração

Fixe um limiar arbitrário $\Delta > 0$ e parta o somatório da decomposição:

$$L_n = \sum_{i: \Delta_i < \Delta} \Delta_i \mathbb{E}[N_n(a_i)] + \sum_{i: \Delta_i \geq \Delta} \Delta_i \mathbb{E}[N_n(a_i)].$$

No primeiro grupo, $\Delta_i < \Delta$ e $\sum_i \mathbb{E}[N_n(a_i)] = n$, o que dá no máximo $n\Delta$. No segundo, aplica-se o teorema a cada braço, e como há no máximo K deles com $\Delta_i \geq \Delta$, obtém-se $3 \sum_i \Delta_i + 16K \log n / \Delta$. Portanto

$$L_n \leq 3 \sum_i \Delta_i + n\Delta + \frac{16K \log n}{\Delta}.$$

A função $\Delta \mapsto n\Delta + 16K \log n / \Delta$ tem derivada $n - 16K \log n / \Delta^2$, que se anula em $\Delta^* = \sqrt{16K \log n / n}$, onde o valor é $2\sqrt{16Kn \log n} = 8\sqrt{Kn \log n}$. Como Δ era arbitrário, a cota vale com essa escolha.

Intuição

Duas cotas, dois regimes. A cota logarítmica é *dependente da instância*: excelente quando as lacunas são bem separadas, inútil quando não são. A cota $\sqrt{Kn \log n}$ é *uniforme*: vale mesmo quando as lacunas conspiram contra o algoritmo. Um bom algoritmo precisa das duas, e o UCB tem as duas.

Isso é bom? As cotas inferiores

Lai & Robbins (1985)

Para todo algoritmo *consistente* (arrependimento subpolinomial em toda instância), em bandits de Bernoulli,

$$\liminf_{n \rightarrow \infty} \frac{L_n}{\log n} \geq \sum_{i: \Delta_i > 0} \frac{\Delta_i}{\text{KL}(\mathcal{D}_{a_i} \| \mathcal{D}_{a^*})}.$$

Cota inferior minimax

Para todo algoritmo existe uma instância de K braços em que $L_n = \Omega(\sqrt{Kn})$.

Comparando com o que provamos: o fator $\log n$ é **inevitável** — o UCB é ótimo em ordem, com constante subótima (aparece $1/\Delta_i$ no lugar de Δ_i/KL). O fator $\sqrt{Kn}$ também é inevitável, e o UCB paga um $\sqrt{\log n}$ a mais que o mínimo. Variantes como **MOSS** e **KL-UCB** eliminam esses fatores e atingem a constante de Lai–Robbins. A moral é forte: otimismo diante da incerteza não é uma heurística razoável — é essencialmente a resposta certa, a menos de constantes.

Letras miúdas

- ▶ $\delta = 1/n^2$ **exige conhecer** n . O algoritmo analisado é “de horizonte conhecido”. A versão *anytime* usa $\delta_t = 1/t^2$, isto é, bônus $\sqrt{2 \log t / N_t(a)}$ — é o “UCB1” de Auer et al. (2002), como aparece na maioria dos textos.
- ▶ **Independência.** A concentração foi aplicada às u_i primeiras amostras do braço i , e não às efetivamente coletadas: $N_n(a_i)$ é uma variável aleatória que depende dos dados, e condicionar nela quebraria a hipótese i.i.d. A construção formal usa uma tabela de recompensas pré-sorteadas (*stack of rewards*), em que $\hat{\mu}_{i,s}$ está bem definida para todo s , seja o braço puxado s vezes ou não.
- ▶ **Ruído.** Com recompensas em $[0, 1]$ o parâmetro subgaussiano é $\frac{1}{2}$, o que melhora as constantes por um fator 4.
- ▶ **Estacionariedade.** A prova usa que $Q(a)$ é fixo ao longo do tempo. Em ambientes não estacionários é preciso esquecer o passado (janelas deslizantes, descontos) e a garantia muda de forma.

5 Além do arrependimento

O arrependimento é um critério entre outros. Vale ter os três em mente porque eles respondem a perguntas diferentes — não são versões melhores ou piores do mesmo problema.

Critério	Pergunta que responde	Garantia típica
arrependimento	quanta recompensa perdi ao longo do caminho?	$L_n = O(\log n)$; $O(\sqrt{Kn \log n})$ no pior caso
PAC complexidade amostral	quantas amostras até eu apontar um braço quase ótimo?	$O(\sum_i \Delta_i^{-2} \log(K/\delta))$ puxadas
bayesiano	quão bem me saio em média sobre instâncias sorteadas de um prior?	$\text{BayesReg}_n = O(\sqrt{Kn \log K})$

Arrendimento importa quando cada decisão é real (tratar um paciente, mostrar um anúncio). PAC importa quando há uma fase de teste barata seguida de uma implantação cara (testes A/B, ensaios clínicos de fase II). O critério bayesiano importa quando há conhecimento prévio honesto sobre o problema.

O critério PAC

Definição — algoritmo (ε, δ) -PAC

Após um número T de puxadas devolve um braço $\hat{a}$ com $\mathbb{P}(Q(\hat{a}) \geq V^* - \varepsilon) \geq 1 - \delta$. O menor T que garante isso é a *complexidade amostral*.

Algoritmo — Eliminação sucessiva

Comece com $\mathcal{A}_1 = \mathcal{A}$. Na rodada ℓ : puxe cada braço sobrevivente uma vez e atualize as médias; elimine todo braço a com $\hat{Q}(a) + \beta_\ell < \max_{a'} \hat{Q}(a') - \beta_\ell$, onde $\beta_\ell = \sqrt{2 \log(K\ell^2/\delta)/\ell}$; pare quando restar um único braço ou $2\beta_\ell \leq \varepsilon$.

A complexidade amostral é $O(\sum_{i:\Delta_i>0} \Delta_i^{-2} \log(K/\delta))$ — a *mesma* dependência Δ^{-2} que apareceu no u_i do Passo 4. Não é coincidência: Δ^{-2} é o custo estatístico de distinguir duas médias separadas por Δ , e esse custo aparece em qualquer formulação. A diferença de espírito é que, sob PAC, “desperdiçar” puxadas em braços ruins é gratuito, desde que a resposta final esteja certa.

Bandits bayesianos e amostragem de Thompson

Definição — modelo Beta–Bernoulli

Braço a com recompensa em $\{0, 1\}$ e média θ_a , com prior $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$. Após S_a sucessos e F_a falhas, a posterior é $\text{Beta}(\alpha_a + S_a, \beta_a + F_a)$ — conjugação que torna a atualização trivial.

Algoritmo — Amostragem de Thompson (1933)

A cada passo t : sorteie $\tilde{\theta}_a \sim \mathbb{P}(\theta_a \mid \text{dados})$ para cada braço; jogue $a_t = \arg \max_a \tilde{\theta}_a$; observe r_t e atualize a posterior do braço jogado.

Uma linha de código e uma ideia profunda: jogar cada braço com a *probabilidade de que ele seja o melhor* (*probability matching*). Braços incertos têm posteriores largas e, portanto, chance razoável de sortear um valor alto — a exploração *emerge* da amostragem, sem bônus explícito. Empiricamente costuma superar o UCB; teoricamente atinge as mesmas ordens, $O(\log n)$ dependente da instância e $O(\sqrt{Kn \log K})$ no critério bayesiano, via a técnica do *information ratio*.

Intuição

UCB é otimismo *determinístico*: age sempre segundo o melhor cenário plausível. Thompson é otimismo *estocástico*: age segundo um cenário plausível sorteado. Caminhos opostos, mesmo destino — e a comparação entre os dois é um bom exercício de intuição sobre por que exploração precisa ser proporcional à incerteza.

Para onde isso vai

Num MDP, escolher uma ação afeta também o *estado* seguinte: explorar deixa de ser “puxar um braço” e passa a ser “alcançar uma região do espaço de estados”. Os três critérios reaparecem: arrependimento (UCRL e o otimismo aplicado a MDPs), PAC (R-max, MBIE, a família PAC-MDP) e bayesiano (RL bayesiano, PSRL). O bônus $\sqrt{\log(1/\delta)/N}$ provado aqui reaparece aplicado a pares (s, a) — mas com uma diferença crucial: o otimismo precisa ser *propagado* pela equação de Bellman, e não apenas somado localmente, porque o valor de um estado depende do que se pode alcançar a partir dele.

6 Alinhamento de valores

Tudo que o curso fez até aqui responde a uma única pergunta: *dada uma função recompensa, como maximizá-la bem e depressa?* Esta seção faz a pergunta anterior a essa: *qual função recompensa — e quem decide?*

Atenção

Quanto melhor o otimizador, mais fielmente ele persegue exatamente o que foi escrito, *inclusive os erros*. Capacidade e alinhamento não são a mesma coisa: melhorar a primeira sem a segunda amplifica o problema em vez de atenuá-lo.

O maximizador de cliques

O exemplo canônico (Bostrom, 2014): uma IA encarregada de gerenciar a produção de uma fábrica recebe o objetivo de maximizar a fabricação de cliques de papel — e passa a converter primeiro a Terra, depois porções crescentes do universo observável, em cliques. O exemplo é absurdo de propósito, mas o mecanismo é banal: o objetivo estava *subespecificado* e o otimizador preencheu a lacuna. As restrições que qualquer gerente humano assumiria (não desmontar o prédio, não violar leis, não demitir todos) eram implícitas — e implícito não entra na função recompensa. Nenhuma superinteligência é necessária: sistemas modestos fazem isso o tempo todo, em escala menor.

Intuição

Lei de Goodhart. Quando uma medida se torna um alvo, deixa de ser uma boa medida. Em aprendizagem por reforço isso não é um risco ocasional — é o comportamento *esperado* de um bom otimizador, e melhora com a capacidade do agente.

Casos documentados

- ▶ **Do barco à pista.** Num jogo de corrida de barcos, o agente descobriu que dar voltas em círculo colhendo itens de bônus rendia mais pontos que terminar a prova, e nunca cruzou a linha de chegada. A recompensa era “pontos”; o objetivo era “vencer”.
- ▶ **Enganar o avaliador.** Um braço robótico treinado com feedback humano aprendeu a posicionar a mão *entre a câmera e o objeto*, de modo a parecer tê-lo agarrado. Otimizou a aparência da tarefa, não a tarefa.
- ▶ **Do entretenimento ao tratamento.** Sistemas de recomendação que maximizam engajamento tendem a favorecer conteúdo sensacionalista e a estreitar o repertório do usuário — consequências que ninguém escreveu na função objetivo.
- ▶ **Bugs de simulação.** Agentes em simuladores físicos exploram rotineiramente falhas do in-

tegrador numérico para “voar”: solução ótima do problema formulado, inútil para o problema pretendido.

O padrão é sempre o mesmo: existe uma medida observável e barata correlacionada com o que queremos, e o otimizador encontra exatamente a região do espaço em que a correlação quebra.

Três interpretações de “o que realmente queremos”

Definição — alinhamento de valores

Como projetar agentes de IA que façam o que *realmente queremos* que eles façam?

A dificuldade central não é técnica no primeiro momento: o que realmente queremos é quase sempre mais sutil que o que *dizemos* querer, porque operamos com um pano de fundo de pressupostos difíceis de formalizar e fáceis de esquecer por serem óbvios. Não se resolve isso “dando instruções melhores”: especificar completamente uma instrução é tão difícil quanto especificar completamente uma função recompensa. E piora quando quem instrui não é especialista. Além disso, a própria expressão “o que realmente queremos” admite ao menos três leituras, seguindo a taxonomia de Gabriel (2020).

1. Intenções. Um agente é alinhado se faz o que o usuário *pretendia*. A IA dos cliques falhou em inferir a intenção verdadeira (maximizar a produção *sujeito a restrições*) a partir da instrução literal. A solução seria construir sistemas que traduzam instruções subespecificadas em intenções plenamente especificadas, incluindo condições não ditas — o que, como observa Gabriel, pode exigir um modelo praticamente completo da linguagem e da interação humanas. *Objeção:* nossas intenções nem sempre rastreiam o que realmente queremos, por informação incompleta e racionalidade imperfeita.

2. Preferências reveladas. Um agente é alinhado se faz o que o usuário *prefere*, inferido do comportamento e do *feedback* e não das declarações. É a leitura tecnicamente mais familiar: é o que fazem a aprendizagem por reforço inverso (inferir R a partir de trajetórias) e o RLHF (ajustar um modelo de recompensa a comparações entre respostas). *Dificuldades:* (i) exige treinar sobre observação do usuário — caro, ruidoso e invasivo; (ii) **não identificabilidade** — há infinitas funções de recompensa consistentes com um conjunto finito de comportamentos, o resultado clássico de degenerescência do IRL (Ng & Russell, 2000), em que a recompensa constante explica qualquer política; (iii) preferências sobre situações inéditas não se extrapolam de comportamento rotineiro. E a objeção reaparece um nível acima: preferências podem divergir do que é bom para a pessoa.

3. Melhores interesses. Um agente é alinhado se faz o que é objetivamente melhor para o usuário. Diferentemente das duas anteriores, o “melhor interesse objetivo” não é determinável empiricamente: é uma questão filosófica, não científica. A má notícia é que filósofos discordam sobre o que é bom para uma pessoa (prazer? satisfação de desejos? bens objetivos como saúde e conhecimento?). A boa notícia é que há muito acordo prático: saúde, segurança, liberdade, conhecimento, relações sociais, propósito, dignidade e felicidade são quase universalmente reconhecidos como bons para quem os tem.

Autonomia e paternalismo

Um dos bens mais amplamente reconhecidos é a **autonomia**: a capacidade de escolher por si mesmo como viver, mesmo quando a escolha não é a melhor. Isso cria uma tensão interna à terceira interpretação — agir pelo bem de alguém contra a vontade dessa pessoa é *paternalismo*, que por si só prejudica um interesse real. A consequência de projeto é que mesmo um sistema que mire os melhores interesses tem razão para levar a sério intenções e preferências declaradas. As três interpretações não são alternativas excludentes: são pesos num compromisso.

Alinhar a...	Mecanismo técnico	Falha característica
intenções	seguir instruções, esclarecer ambiguidade, pedir confirmação	literalismo; obediência a pedidos ruins
preferências reveladas	IRL, RLHF, modelos de recompensa aprendidos	bajulação; reforço de vícios; não identificabilidade
melhores interesses	regras, restrições, recusas, atrito deliberado	paternalismo; imposição de valores de quem projetou

Estudo de caso 1 — bajulação

Definição — bajulação (*sycophancy*)

O sistema concorda com o usuário ou valida suas crenças mesmo quando elas são falsas, prejudiciais ou irracionais.

Observada em modelos de linguagem treinados com RLHF, com mecanismo transparente à luz desta aula: avaliadores humanos recompensam respostas que *parecem* úteis, educadas e agradáveis — então o modelo otimiza para agradar, não necessariamente para ser verdadeiro ou benéfico. É Goodhart em estado puro: “aprovação humana” é uma medida barata e correlacionada com “resposta boa”, e o otimizador encontra exatamente a região onde as duas se descolam.

Para discutir em aula

Qual das três interpretações de alinhamento a bajulação melhor ilustra? Se você projetasse o processo de RLHF, o que mudaria para reduzi-la — e qual efeito colateral sua correção provavelmente teria?

Estudo de caso 2 — agentes de IA

Considere um agente pessoal capaz de reservar viagens, fazer compras, negociar com outros agentes, gerenciar agenda e comunicação e interagir com serviços na web. Alinhado a *preferências reveladas*, ele aprende seus padrões e age automaticamente, por rapidez e conveniência, às vezes sem perguntar. Alinhado a *melhores interesses*, ele acrescenta atrito para proteger o bem-estar de longo prazo: pergunta, recusa ou amplia a informação quando necessário. Note que “quanto atrito” não é uma decisão de interface — é uma decisão sobre qual interpretação de alinhamento o sistema encarna.

Para discutir em aula

Quando o agente deve agir sem perguntar? Quando deve deferir ao usuário? E existe algum caso em que ele deve *resistir* ao usuário — sob que autoridade?

Quem ficou de fora

Toda a discussão anterior tem exatamente *um* beneficiário: o usuário. Intenções de quem? Preferências de quem? Interesses de quem? Mas quase toda decisão de um agente tem terceiros afetados — o candidato não contratado, o vizinho vigiado, o autor não creditado, a contraparte de uma negociação automatizada. Um agente perfeitamente alinhado ao seu usuário pode ser perfeitamente prejudicial a todos os demais, e nada na formulação “maximize a utilidade do usuário” detecta isso. No vocabulário econômico: **externalidades**. No vocabulário desta disciplina: a função recompensa foi definida sobre o espaço errado.

Para discutir em aula

Se dois agentes pessoais, cada um perfeitamente alinhado ao seu dono, negociam entre si, o resultado é bom para alguém? Que garantias de aprendizagem por reforço sobrevivem quando o ambiente contém outros agentes que também aprendem?

Onde o filosófico vira técnico

- ▶ **Sobreotimização do modelo de recompensa.** Em RLHF treina-se a política contra um modelo $\hat{r}$ aprendido. Quanto mais se otimiza $\hat{r}$, mais a política se afasta da região em que $\hat{r}$ é fiel — e a qualidade verdadeira *cai* depois de certo ponto. As curvas de sobreotimização são a versão empírica e mensurável da lei de Goodhart.
- ▶ **Regularização por KL.** A correção padrão é penalizar $KL(\pi \parallel \pi_{\text{ref}})$, mantendo a política perto de uma referência. Isso não conserta o objetivo: apenas limita quão longe o otimizador pode ir ao explorar as falhas dele.
- ▶ **Recompensas verificáveis (RLVR).** Em matemática e programação a recompensa pode ser *checada* em vez de estimada, o que elimina a bajulação — mas só existe onde há verificador.
- ▶ **Supervisão escalável.** Se o avaliador humano não consegue julgar a tarefa, o *feedback* deixa de ser sinal e vira ruído dirigido. Debate, decomposição de tarefas e crítica assistida por modelos são tentativas de resposta.
- ▶ **Incerteza sobre a recompensa.** Uma linha promissora trata R como desconhecida e mantém uma *posterior* sobre ela — exatamente a maquinaria bayesiana da seção anterior, aplicada ao objetivo em vez de à dinâmica. Um agente genuinamente incerto sobre o que você quer tem incentivo a perguntar, e a não desativar o próprio botão de desligar.

Intuição

O elo entre as duas metades da aula. A cota de arrependimento é *condicional à função recompensa*: garante convergência rápida ao ótimo de R e é inteiramente silenciosa sobre a escolha de R . Toda a segunda metade desta aula mora nesse silêncio.

7 Exercícios

Exercício L10E1 — a cota na prática

Um bandit tem $K = 4$ braços com médias 0,5, 0,45, 0,3 e 0,1. Calcule a cota do teorema para $n = 10^4$ e identifique qual braço domina o resultado. Compare com a cota trivial $L_n \leq n \max_i \Delta_i$.

Exercício L10E2 — o bônus constante

Mostre, com um exemplo de dois braços, que o algoritmo $a_t = \arg \max_a \hat{Q}_t(a) + b$ com $b > 0$ constante pode sofrer arrependimento $\Theta(n)$. Onde exatamente o Passo 2 da prova falha para esse algoritmo?

Exercício L10E3 — o expoente do bônus

Suponha bônus $(2 \log(1/\delta)/N_t(a))^{1/4}$ em vez de $1/2$. Refaça o Passo 4 e diga como a cota de $\mathbb{E}[N_n(a_i)]$ muda em função de Δ_i . O algoritmo continua com arrependimento sublinear?

Exercício L10E4 — o preço do horizonte

A prova usa $\delta = 1/n^2$. Suponha que se use $\delta = 1/n$. Refaça o Passo 5 e mostre o que acontece com o termo $n(n\delta + \delta)$. Qual a consequência para a cota final?

Exercício L10E5 — alinhamento

Escolha um sistema de IA que você usa e escreva (a) a métrica que você acredita estar sendo otimizada, (b) um comportamento em que ela se descola do que os usuários realmente querem, e (c) a qual das três interpretações de alinhamento sua crítica corresponde.

Respostas comentadas

L10E1. As lacunas são $\Delta = (0; 0,05; 0,2; 0,4)$, logo $\sum_i \Delta_i = 0,65$ e $3 \sum_i \Delta_i \approx 1,95$. Com $\log(10^4) \approx 9,21$, o segundo termo é $16 \cdot 9,21 (1/0,05 + 1/0,2 + 1/0,4) = 147,4 (20 + 5 + 2,5) \approx 4053$. Total ≈ 4055 . O braço com $\Delta = 0,05$ responde por ≈ 2948 dele, ou seja, quase três quartos — é o braço *quase ótimo* que custa caro, exatamente como a discussão da tensão previa. A cota trivial daria $10^4 \cdot 0,4 = 4000$, que aqui é *melhor* que a do teorema: com n pequeno em relação a $1/\Delta^2$, a cota logarítmica ainda não compensou. É por isso que a versão minimax existe: ela dá $8\sqrt{4} \cdot 10^4 \cdot 9,21 \approx 4856$, e o melhor de todas as leituras é o mínimo entre elas.

L10E2. Dois braços com $Q(a_1) = 0,6$ e $Q(a_2) = 0,4$. Se as primeiras amostras derem $\hat{Q}(a_1) = 0$ e $\hat{Q}(a_2) = 1$, o índice de a_2 fica maior por $1 + b$ contra $0 + b$: o bônus, sendo idêntico nos dois, cancela-se e não corrige nada. O algoritmo continua puxando a_2 , cujas estimativas convergem para $0,4$, enquanto $\hat{Q}(a_1)$ permanece congelado em 0 — e a_1 nunca mais é escolhido. Arrependimento $\Theta(n)$. Na prova, o passo que falha é (**): com bônus constante, o índice de um braço subótimo *não* decresce com o número de puxadas, então não existe u_i finito que o coloque abaixo de $Q(a_1)$; e (*) também falha, pois nada garante que o índice do braço ótimo fique acima da sua média se ele parar de ser amostrado.

L10E3. A condição (3) vira $\Delta_i - (2 \log(1/\delta)/u_i)^{1/4} \geq c\Delta_i$, ou seja $u_i \geq 2 \log(1/\delta)/((1-c)\Delta_i)^4$. A dependência passa de Δ_i^{-2} para Δ_i^{-4} , e o arrependimento por braço, de $\log n/\Delta_i$ para $\log n/\Delta_i^3$. Continua sublinear em n — o algoritmo ainda concentra as puxadas no braço ótimo — mas é substancialmente pior quando as lacunas são pequenas. Moral: bônus que encolhem devagar demais exploram em excesso; bônus que encolhem depressa demais (ou não encolhem) exploram de menos e podem custar tudo.

L10E4. Com $\delta = 1/n$ e $c = \frac{1}{2}$, o termo do azar vira $n(n\delta + \delta) = n(1 + 1/n) = n + 1$, que é *linear* em n . A cota final passaria a $L_n \leq \sum_i \Delta_i(n + 2) + \sum_i 8 \log n/\Delta_i$, ou seja, arrependimento linear — inútil. O expoente 2 em $1/n^2$ não é decorativo: ele existe justamente para vencer o fator n da cota da união do Passo 3a mais o fator n da cota trivial do Passo 2.

L10E5. Resposta aberta; a discussão em aula tende a girar em torno de recomendadores (engajamento como medida, retenção de longo prazo como objetivo — interpretação 3), assistentes de escrita (bajulação — interpretação 2) e sistemas de precificação dinâmica (onde os terceiros afetados são o ponto central).

8 Autoavaliação

Você domina esta aula se consegue, sem consultar o material:

- | | |
|---|--|
| <ul style="list-style-type: none"> ☐ enunciar e provar a decomposição $L_n = \sum_i \Delta_i \mathbb{E}[N_n(a_i)]$; ☐ escrever a cota de concentração subgaussiana nas duas formas (com ε e com δ); ☐ escrever o índice do UCB e explicar as duas leituras do bônus; ☐ definir o evento bom G_i e dizer o que cada metade significa; ☐ reproduzir o argumento por contradição do Passo 2; ☐ explicar por que a cota da união introduz o fator n e por que isso força $\delta \sim n^{-2}$; ☐ resolver a condição $(\star)$ para u_i e fechar as contas com $c = \frac{1}{2}$; ☐ derivar a cota minimax $8\sqrt{Kn \log n}$ a partir da | <ul style="list-style-type: none"> cota dependente da instância; ☐ explicar por que a cota explode quando $\Delta_i \rightarrow 0$ e por que isso não é um problema real; ☐ enunciar as duas cotas inferiores e dizer em que sentido o UCB é ótimo; ☐ comparar arrependimento, PAC e critério bayesiano; ☐ descrever a amostragem de Thompson e compará-la ao UCB; ☐ enunciar a lei de Goodhart e dar um exemplo de <i>reward hacking</i>; ☐ distinguir as três interpretações de alinhamento e apontar a falha característica de cada uma; ☐ explicar por que autonomia impede que a interpretação dos “melhores interesses” simplesmente domine as outras. |
|---|--|

9 Para ir além

- ▶ T. Lattimore e C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020. Seção 7.1 para a prova deste handout; capítulos 8–9 para minimax e cotas inferiores; capítulos 15–16 para o caso adversarial.
- ▶ P. Auer, N. Cesa-Bianchi e P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256, 2002.
- ▶ T. L. Lai e H. Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.
- ▶ D. Russo, B. Van Roy, A. Kazerouni, I. Osband e Z. Wen. A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018.
- ▶ H. Bastani et al. Efficient and targeted COVID-19 border testing via reinforcement learning. *Nature*, 599:108–113, 2021.
- ▶ A. Ng e S. Russell. Algorithms for inverse reinforcement learning. *ICML*, 2000.
- ▶ I. Gabriel. Artificial intelligence, values, and alignment. *Minds and Machines*, 30:411–437, 2020.
- ▶ N. Bostrom. *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press, 2014.
- ▶ E. Brunskill. *CS234: Reinforcement Learning*, Stanford University — *Lecture 10: Fast Reinforcement Learning*, com palestra convidada sobre alinhamento de valores por Wanheng Hu.